POE Part 3: Manual Steps

CLDV6212 CoffeeNChill. Everything you must do yourself to get the finished code live, recorded and submitted.

Start here

The code, Dockerfiles, compose file, pipeline, Postman collection and README are done. What is left needs your own accounts: Docker Hub, GitHub, a VPS and YouTube. Work through the sections in order, because each one uses the result of the one before it.

#	Section	Result
1	Install the tools	Docker Desktop, Git, Postman and Storage Explorer ready
2	Test on your PC	Stack running locally and Postman passing
3	Push to GitHub	Public repository with your commits
4	Push to Docker Hub	`v3.0` images published
5	Create the VPS	Ubuntu server with port 5000 open
6	Deploy on the VPS	Public Swagger URL working
7	Finish the README	Live URL in the repository
8	Bonus CI/CD pipeline	Push to main deploys automatically (+10 marks)
9	Record the video	Unlisted YouTube link
10	Submit	Every deliverable checked

What has already been tested, and what has not

Tested on this PC and passed	Could not be tested here: please confirm in the steps below
Gateway and Functions built with .NET 10 in Release mode. 13 unit tests passed. Gateway run in Production mode against the real Functions host and Azurite. Full Postman suite: 46 requests, 84 assertions, 0 failures. Passwords saved as BCrypt hashes. JWT claims correct. Order reached Collected. Swagger padlocks match the POE.	Docker is not installed on this PC, so the two image builds and `docker compose` have not been run (the YAML files parse correctly). The function key check only happens inside the container, so Section 2 is the first real test of it. The VPS and GitHub Actions need your accounts, so they have not been run.

The accounts in the system

Role	How it is created	Can do
Customer (student)	Anyone, with `POST /api/auth/register`	Same as public. Blocked (403) from staff endpoints
Staff (barista)	Registered using an Admin token	Change the menu, files and order statuses
Admin (manager)	Created at startup from `.env`	Same as Staff, and can create Staff and Admin users

1. Install the tools

1. Install **Docker Desktop**: https://www.docker.com/products/docker-desktop/ . Open it and wait until it says it is running.
2. Install **Git**: https://git-scm.com/download/win
3. Install **Postman**: https://www.postman.com/downloads/
4. Install **Azure Storage Explorer**: https://azure.microsoft.com/products/storage/storage-explorer/
5. Open a new PowerShell window and confirm that these all print a version:

```
docker --version
docker compose version
git --version
```

2. Test on your PC

Section 2 uses the Part 3 folder, `cldv6212-g3-2026-poe-part3-group-22`. Leave the Part 2 folder as it is.

1. Open PowerShell in the folder that contains `docker-compose.yml` and `README.md`.
2. Create your settings file and open it:

```
Copy-Item .env.example .env
notepad .env
```

3. Generate a JWT key in a second PowerShell window, then paste it after `JWT_KEY=` in Notepad:

```
-join ((1..32) | ForEach-Object { '{0:x2}' -f (Get-Random -Maximum 256) })
```

4. Change `ADMIN_PASSWORD` to a password of your own. Do not use `$`, `#` or quotes in it. Save and close Notepad.
5. Build both images and start the stack. The first build takes a few minutes:

```
docker build -t bonolo05/coffeenchill-functions:v3.0 ./CoffeeNChill
docker build -t bonolo05/coffeenchill-gateway:v3.0 ./CoffeeNChill.Gateway
docker compose up -d
docker compose ps
```

6. All three containers must show as running. Open http://localhost:5000/swagger and check that the padlocks appear only on protected endpoints.
7. In Postman choose **Import** and select these files from `CoffeeNChill/docs`: `CoffeNChill.postman_collection.json`, `CoffeNChill.local.postman_environment.json` and `CoffeNChill.vps.postman_environment.json`.
8. Select the **CoffeNChill Local** environment, open it and type your Admin password as the value of `adminPassword`. Save.
9. Click the collection, choose **Run**, keep the default order and start the run. All tests must pass. If the first requests give 503, the Functions host was still starting, so wait 20 seconds and run again.

> **If the Menu or Documents requests give 502**, the function key does not match. Check that `FUNCTION_KEY` in `.env` matches the `default` value in `docker/keys/host.json`, then run `docker compose up -d` again.

3. Push to GitHub

> **The repository must be public, or follow your lecturer's GitHub Classroom instructions.** The rubric deducts 5 marks for a private repository or for not using GitHub correctly. The rubric also checks commit history, so if you are in a group, each member should commit the work they did. Do not change commit dates or authors.

1. Accept your Part 3 GitHub Classroom link, or create a new repository on GitHub. Copy its HTTPS address.
2. Clone it into a new folder, for example `Documents\part3`.

```
cd $HOME\Documents
git clone https://github.com/YOUR-ACCOUNT/YOUR-REPOSITORY.git part3
cd part3
git branch -M main
```

3. **Push your Part 2 code first.** In the same PowerShell window, copy the Part 2 folder into `part3`, commit it and push it. Your unchanged Part 2 becomes the first commit, so the history shows exactly what Part 3 added.

```
$o = "$HOME\Downloads\cldv6212-g3-2026-poe-part2-group-22-main"
robocopy "$o\cldv6212-g3-2026-poe-part2-group-22-main" . /E /XD bin obj .vs
git add .
git commit -m "Start from Part 2 code"
git push -u origin main
```

4. **Now bring in Part 3.** Delete everything in `part3` except `.git`, then copy the Part 3 folder in. Deleting first means files removed in Part 3, such as `Function1.cs`, also show as removed. Your Part 2 is safe in the first commit and in its own folder.

```
Get-ChildItem -Force | Where-Object { $_.Name -ne ".git" } |
  Remove-Item -Recurse -Force
robocopy "$o\cldv6212-g3-2026-poe-part3-group-22" . /E /XD bin obj .vs
```

5. Confirm that your secrets file is ignored. This command must print `.env`:

```
git check-ignore .env
```

6. Temporarily keep the pipeline out of the first pushes, because it would fail until Section 8 is set up. Commit in logical parts:

```
git add CoffeeNChill docker docker-compose.yml .env.example .gitignore
git commit -m "Add order status and document delete functions, update compose for VPS"

git add CoffeeNChill.Gateway
git commit -m "Add API gateway with JWT auth, DTOs, Swagger and error middleware"

git add CoffeeNChill.Tests
git commit -m "Add gateway unit tests"

git add README.md
git commit -m "Update README for Part 3"

git push -u origin main
```

7. Open the repository on GitHub and confirm that `.env` is **not** listed.

> Run `git status` before the push. `.github/` should be the only item left untracked at this point.

4. Push to Docker Hub

1. Sign in, using your `bonolo05` Docker Hub account:

```
docker login -u bonolo05
```

2. Push the images you built in Section 2:

```
docker push bonolo05/coffeenchill-functions:v3.0
docker push bonolo05/coffeenchill-gateway:v3.0
```

3. Open https://hub.docker.com/u/bonolo05 and confirm that both repositories have the `v3.0` tag and are **Public**. The VPS pulls them without logging in.

> If you change any code later, build and push the images again before updating the VPS. Once the Section 8 pipeline is working, it does this for you.

5. Create the VPS

The steps below use an **Azure VM**, because an Azure for Students subscription can pay for it. Any Linux VPS works if it has Ubuntu on an x64 processor and at least 2 GB of RAM (DigitalOcean, AWS EC2 or Linode). If you use another provider, create the server, open ports 22 and 5000, and continue from Section 6.

> Avoid ARM servers such as a Raspberry Pi or AWS Graviton, because these images are built for x64. Hosts like Render do not run a `docker compose` stack like a VPS does, so use a real Linux server.

1. Go to https://portal.azure.com , open **Virtual machines**, then **Create > Azure virtual machine**.
2. Fill in the **Basics** tab:

Setting	Value
Subscription	Azure for Students
Resource group	Create new: `coffeenchill-rg`
Virtual machine name	`coffeenchill-vps`
Region	South Africa North. If the size or region is refused, try another region your subscription allows
Image	Ubuntu Server 24.04 LTS - x64 Gen2
Size	Standard_B2s (4 GB RAM) is best. Standard_B1ms (2 GB) is the minimum. Do not use a 1 GB size
Authentication type	SSH public key
Username	`azureuser`
SSH public key source	Generate new key pair
Key pair name	`coffeenchill-vps_key`
Public inbound ports	Allow selected ports: SSH (22)

3. Click **Review + create**, then **Create**. When prompted, click **Download private key and create resource**.

4. When the deployment finishes, open the VM and write down its **Public IP address**. The rest of this guide calls it `YOUR-VPS-IP`.
5. Open **Network settings** on the VM, choose **Create port rule > Inbound port rule** and enter: Destination port ranges `5000`, Protocol `TCP`, Action `Allow`, Name `Allow-Gateway-5000`. Click **Add**.
6. On your PC, move the key to a safe place and lock its permissions:

```
New-Item -ItemType Directory -Force "$HOME\.ssh"
Move-Item "$HOME\Downloads\coffeenchill-vps_key.pem" "$HOME\.ssh\"
icacls "$HOME\.ssh\coffeenchill-vps_key.pem" /inheritance:r /grant:r "$($env:USERNAME):(R)"
```

7. Connect to the server. Type `yes` the first time:

```
ssh -i "$HOME\.ssh\coffeenchill-vps_key.pem" azureuser@YOUR-VPS-IP
```

> A running VM uses your credit. When you are not working on it, click **Stop** in the portal so that it shows **Stopped (deallocated)**. Start it again before the video.

6. Deploy on the VPS

Run all of these commands inside the SSH session.

Install Docker Engine and Docker Compose

1. Add Docker's official package source and install it:

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl git
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin \
  docker-compose-plugin
sudo usermod -aG docker $USER
```

2. Type `exit`, connect again with the same `ssh` command, then check the installation:

```
docker run --rm hello-world
docker compose version
```

Start the application

3. Clone your repository into the folder name that the pipeline expects:

```
git clone https://github.com/YOUR-ACCOUNT/YOUR-REPOSITORY.git ~/coffeenchill
cd ~/coffeenchill
```

4. Create the settings file and generate a new JWT key. Do not reuse the one from your PC:

```
cp .env.example .env
sed -i "s/^JWT_KEY=.*/JWT_KEY=$(openssl rand -hex 32)/" .env
nano .env
```

5. In nano, set a new strong `ADMIN_PASSWORD`. Save with `Ctrl+O`, `Enter`, then exit with `Ctrl+X`.

6. Pull the published images and start the stack:

```
docker compose pull
docker compose up -d
docker compose ps
```

7. Wait about 20 seconds and test it from the VPS. It should print `[]` or a list of items:

```
curl http://localhost:5000/api/menu
```

8. On your PC, open `http://YOUR-VPS-IP:5000/swagger`.
9. In Postman open the **CoffeNChill VPS** environment. Set `baseUrl` to `http://YOUR-VPS-IP:5000` and `adminPassword` to the VPS Admin password. Select the environment and run the whole collection. All tests must pass.
10. Check that the stack survives a restart: run `sudo reboot`, wait a minute, reconnect and run `docker compose ps` in `~/coffeenchill`.

> **Security:** this site uses plain HTTP as in the POE's example address, so only use test passwords. The POE allows you to disable public access once the video is complete. Functions and Azurite have no public ports. Keep it that way.

7. Finish the README and Postman file

1. Work in your `part3` folder. In `README.md`, replace `http://YOUR-VPS-IP:5000/swagger` near the top with your real address. Replace `YOUR-ACCOUNT/YOUR-REPOSITORY` in the VPS section. Check that the repository name in the Project structure section matches yours.
2. In `CoffeeNChill/docs/CoffeNChill.vps.postman_environment.json`, replace `YOUR-VPS-IP` with your real IP. Leave `adminPassword` empty.
3. Commit and push these changes. The YouTube link is added in Section 9.

```
git add README.md CoffeeNChill/docs/CoffeNChill.vps.postman_environment.json
git commit -m "Add live VPS address"
git push
```

8. Bonus: CI/CD pipeline (+10 marks)

The workflow file is already written. It builds, runs the unit tests, pushes tagged images and deploys over SSH. You only need to give it the credentials.

Create a deploy key on the VPS

1. Run this on the VPS. Copy the whole private key that it prints, including the `BEGIN` and `END` lines:

```
ssh-keygen -t ed25519 -f ~/.ssh/github_deploy -N ""
cat ~/.ssh/github_deploy.pub >> ~/.ssh/authorized_keys
cat ~/.ssh/github_deploy
```

Create a Docker Hub token

2. On Docker Hub open **Account settings > Personal access tokens > Generate new token**. Give it **Read & Write** access and copy the token.

Add the GitHub secrets

3. In your repository open **Settings > Secrets and variables > Actions > New repository secret**, and add these five:

Secret name	Value
`DOCKERHUB_USERNAME`	`bonolo05`
`DOCKERHUB_TOKEN`	The Docker Hub token
`VPS_HOST`	Your VPS public IP
`VPS_USERNAME`	`azureuser` (or your server's username)
`VPS_SSH_KEY`	The complete private key from step 1

4. Delete the private key file from the VPS. It is now stored as a GitHub secret:

```
rm ~/.ssh/github_deploy
```

Turn on the pipeline

5. Push the workflow file from your PC:

```
git add .github
git commit -m "Add GitHub Actions deploy pipeline"
git push
```

6. Open the **Actions** tab on GitHub and watch **Build and Deploy**. Every step must be green.
7. Check Docker Hub for new tags such as `v3.0.1`. On the VPS, run `docker ps` and confirm the containers use that same tag.

> The pipeline deploys only if your VPS is running and the repository is at `~/coffeenchill`. The POE writes `docker-compose`; current Docker Engine installs use `docker compose` with a space, which does the same thing.

9. Record the video

Follow this order, because it covers each rubric point. Speak over everything and explain why, not only what. Show the address bar with the VPS IP while using Swagger.

Part	What to show and say
1. Architecture	README diagram. The gateway is the only public service. It calls Functions over the private Docker network and the Users table directly.
2. VPS hosting	SSH session: `docker compose ps`, `docker images`. Show the images came from Docker Hub. Show the port 5000 rule. Point out that Functions has no public port.
3. Swagger	Open `/swagger` on the live IP. Show the XML descriptions, response codes and schemas. Show that only protected endpoints have padlocks.
4. Public access	`GET /api/menu` gives 200 with no login. `POST /api/orders/queue` gives 202. Try `POST /api/menu` with no token: 401.
5. Student role	Register a Customer, log in, copy the token, click **Authorize** and paste only the token. `POST /api/menu` now gives 403. Explain 401 versus 403. Click **Logout** in the Authorize window.
6. Staff and Admin	Log in as Admin and Authorize. Register a Staff user with `"role": "Staff"`. Log in as Staff and Authorize. Add a menu item (201), change its price, upload a file, and update an order's status.
7. Hashed passwords	Open the Users table using the steps below. Point out PartitionKey = role, RowKey = email, and that PasswordHash begins with `$2a$`.
8. Postman	Select the VPS environment. Show the `{{token}}` Bearer auth. Run the whole collection and show all tests passing.
9. Pipeline	If you did the bonus: show `deploy.yml`, a green Actions run, and the version tags on Docker Hub.
10. Design choices	Use the talking points below.

See the Users table on the VPS

1. Open a new PowerShell window and keep this tunnel connected. It reaches Azurite without opening any public port:

```
ssh -i "$HOME\.ssh\coffeenchill-vps_key.pem" -L 10000:127.0.0.1:10000 `
  -L 10001:127.0.0.1:10001 -L 10002:127.0.0.1:10002 azureuser@YOUR-VPS-IP
```

> Stop any local Docker stack first with `docker compose down`, because it uses the same ports.

2. In Azure Storage Explorer open **Emulator & Attached > Storage Accounts > (Emulator - Default Ports) > Tables > Users**.

Design choices you should be able to explain

- **Gateway:** clients get one address, and security is enforced in one place before a request reaches the backend.
- **BCrypt:** a slow, salted, one-way hash. A leaked table does not reveal the passwords.
- **JWT:** the server does not store sessions. The signature proves the claims have not been changed, and the token expires after 60 minutes.
- **Admin-only role creation:** if the public register endpoint accepted any role, anyone could become an Admin. So public users can only be Customers.

- **DTOs:** clients only send and receive the intended fields. The password hash and storage keys are not exposed.
- **Exception middleware:** one place handles errors, every error has the same format, and internal details stay hidden.
- **Only port 5000 is public:** the Azurite development key is public knowledge, so storage must never be reachable from the internet.
- `.env`**:** the JWT key and Admin password are not in GitHub. Anyone with the JWT key could forge an Admin token.

Publish it

3. Upload to YouTube with visibility set to **Unlisted**.
4. Add the link in the two `ADD-YOUR-YOUTUBE-LINK-HERE` places in `README.md`, then commit and push.

10. Submit

Check each item against the POE's deliverables and rubric:

- [ ] Repository is accessible to the marker and has source code, Dockerfiles, `docker-compose.yml` and `deploy.yml`.
- [ ] `.env` is not in the repository.
- [ ] Commit history shows real work from every team member.
- [ ] README has the live URL, the Part 3 video link and no leftover `YOUR-` or `ADD-YOUR` text.
- [ ] Docker Hub has public `v3.0` images for gateway and functions.
- [ ] `/swagger` opens from another device or mobile data, not only your own network.
- [ ] Postman collection and the VPS environment are in `CoffeeNChill/docs`, and a full run passed against the VPS.
- [ ] Video is unlisted, has a voiceover and shows live VPS, Swagger JWT flow, Customer versus Staff roles and the Postman run.
- [ ] If your institution requires an AI-use declaration, it is completed and submitted with this POE.

After your mark is confirmed

Delete the `coffeenchill-rg` resource group in the Azure portal so it stops using credit. Check first whether your lecturer needs the site to stay online until marking is finished.

Troubleshooting

Problem	Fix
`Set JWT_KEY in the .env file`	You are in the wrong folder or `.env` does not exist. Run commands next to `docker-compose.yml`.
Gateway keeps restarting	Run `docker compose logs gateway`. A message about `Jwt:Key` means `JWT_KEY` still needs a real value of at least 32 characters.
503 from the gateway	Functions is still starting or has stopped. Wait 20 seconds, then run `docker compose logs functions`.
502 from the gateway	`FUNCTION_KEY` in `.env` does not match `docker/keys/host.json`.
Admin login gives 401 after changing the password in `.env`	The Admin is only created once. To start again, run `docker compose down -v` and `docker compose up -d`. This deletes all stored data.

Problem	Fix
Swagger gives 401 after Authorize	Paste only the token, without the word Bearer. Tokens last 60 minutes, so log in again if needed.
Live URL times out	Check the port 5000 inbound rule and that the VM is running. Then run `curl http://localhost:5000/api/menu` on the VM.
`permission denied` on `docker`	Log out of SSH and connect again after the `usermod` step.
`UNPROTECTED PRIVATE KEY FILE`	Run the `icacls` command from Section 5 again.
`docker compose pull` says denied or not found	Check that the images were pushed and are public, and that `IMAGE_TAG` in `.env` exists on Docker Hub.
VM runs out of memory or is very slow	Resize it to Standard_B2s. Check memory with `free -h`.
Pipeline fails at the SSH step	Check the VPS_ secrets, including the whole private key, and that port 22 allows GitHub. Do not restrict SSH to your own IP if you use the pipeline.
Pipeline `git pull` fails on the VPS	Do not edit tracked files on the VPS. Run `git status` and restore any changed file with `git checkout -- filename`.